MASTER OF BUSINESS ADMINISTRATION

SEMESTER 3

DADS304

VISUALIZATION

Unit 5

Creating Advanced Dashboard and Visualization

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	1	-	3 - 5
	1.1 Objectives	-	-	
2	Creation of Advanced Interactive R-Shiny Dashboard	2 to 19	1	6 - 14
3	Addition Of Advanced Visualizations In R-Shiny Dashboard	20 - 32	2	15 - 23
4	Activity	-	-	24
5	Summary	-	-	24
6	Glossary	-	-	25
7	Concept Map	33 to 34	-	25
8	Study Notes & Did You Know	-	-	26
9	Case Study	35 to 36	-	26 - 27
10	Terminal Questions	-	-	28
11	Self-Assessment Answers	-	-	28
12	Terminal Questions Answers	-	-	28 – 39
13	References	-	-	39

1. INTRODUCTION

What is a dashboard?

Dashboards are tools that offer current information while providing images to explain the information about the data. They assist decision-makers in understanding the connections in complex, massive data. They display images in a useful arrangement that makes it easier for the organization to understand and appreciate the data.

Shiny

The RStudio PBC team created the open source R package known as Shiny. To offer a beautiful and simple web framework for creating online apps in R, RStudio created Shiny. R users can build amazing apps, interactive maps, and dashboards using Shiny. And to construct it, you don't need significant web development abilities!

Why Do We Need a Database?

- Timely information - Compress and organize important information. Assisting in the identification of the most effective responses to queries connected in decision-making.
- Access critical Insights- Follow the trends in sales data and KPIs, which are all derived from different data sources.
- Increase performance - It provides a window for trend visualization and business metric tracking.
- Customization and scale - Adapt to the requirements of certain user roles, departmental demands, or a complete organization. Boost operational effectiveness while preserving brand coherence.
- Collaboration - Create a user interface for multi-user collaboration. Encourage collaboration among teams.

Why do we build dashboards in R Shiny?

Access to a full web application framework is made possible within the R environment via shiny dashboards. You can quickly create web applications that benefit organizations from your R work, including analyses, visualizations, machine learning models, and more. End-users can use it as a complete application without knowing how to use R. Deliver a comprehensive, user-friendly, and interactive product that enhances how you conduct business.

Custom Shiny dashboards

The dashboard may be easily customized using unique HTML, CSS, SCSS, Javascript, and other languages thanks to Shiny's web framework. With other BI software suites, it is impossible to develop a distinctive, customized dashboard with this level of flexibility. Include elements like colors, logos, typefaces, and others that better reflect your company.

Cost

Comparing Shiny to competitors like Power BI and Tableau, the latter two are less expensive and open source. On the Appsilon blog, you can examine a detailed comparison of Shiny to Power BI and Shiny to Tableau.

Open Source and Accessibility

One may use Shiny as a dashboard development platform to access a variety of R packages for data research, including the Tidyverse. For the visualization of data and models, you can access advanced graphical features. Add reactivity and interaction by embedding these images in Shiny dashboards. This can be done by using an interface that R has allowed to communicate with JavaScript-based charting packages.

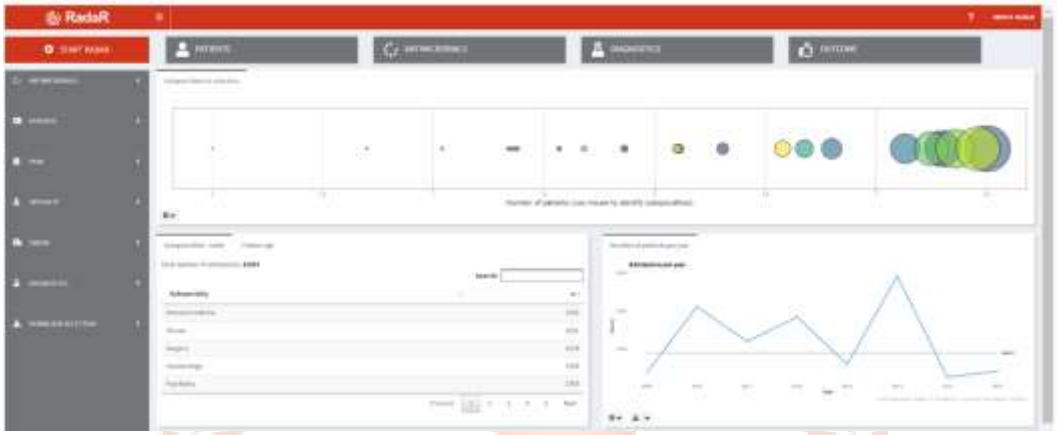


Fig 1: R Shiny Dashboard for Healthcare

1.1 Objectives

You ought to be able to after studying this chapter :

- *Create Advanced Interactive R-Shiny Dashboard*
- *Addition of Advanced Visualization in R-Shiny dashboard*

INSPIRED BY LIFE

2. CREATION OF ADVANCED INTERACTIVE R-SHINY DASHBOARD

R Shiny is a fantastic tool for quickly producing aesthetically stunning and practical dashboards, and it's not too difficult to master. Deploying more sophisticated solutions, nevertheless, faces two significant obstacles.

First off, without a basic understanding of CSS and JavaScript, implementing unique dashboard designs could prove challenging. Second, the process of putting the software into production also calls for more sophisticated expertise. There isn't much of a problem because my app is never used by more than two people at once. It does need more steps, though, to make it accessible to a few hundred individuals.

Contents involved to create a Shiny Dashboard

- Data sources
- Data transformation
- Shiny App structure
- Visualizations
- Dashboard deployment on the shinyapps.io website server.

Let us get to know each topic in detail with an example

1. Data Source:-

In this study, two datasets will be used.

➔ Data from the Graduate Employment Survey is available at data.gov.sg/ges.

	school	degree	employment_rate_overall	employment_rate_ft_perm
1	College of Engineering	Aerospace Engineering	95.3	95.3
2	College of Engineering	Bioengineering	81.3	68.8
3	College of Engineering	Chemical and Biomolecular Engineering	87.3	85.1
4	College of Engineering	Computer Engineering	90.3	88.2
5	College of Engineering	Civil Engineering	94.8	93.8
6	College of Engineering	Computer Science	92.1	88.5

6 rows | 1-5 of 5 columns

Fig 2: Graduate employment dataset

➔ Users can find the dataset for grads by university here: data.gov.sg/gra.

```
  year sex                      university graduates
1 1983  MF National University of Singapore      2905
2 1983   F National University of Singapore      1508
3 1984  MF National University of Singapore      3409
4 1984   F National University of Singapore      1826
5 1985  MF National University of Singapore      3454
6 1985   F National University of Singapore      2117
```

Fig 3: Grads Dataset by university

As previously indicated, the analysis's primary metrics are:

- Rate of Employment
- Gross Monthly Income

The R packages used are

Fig 4: R packages used in creating the dashboard

2. Data transformation

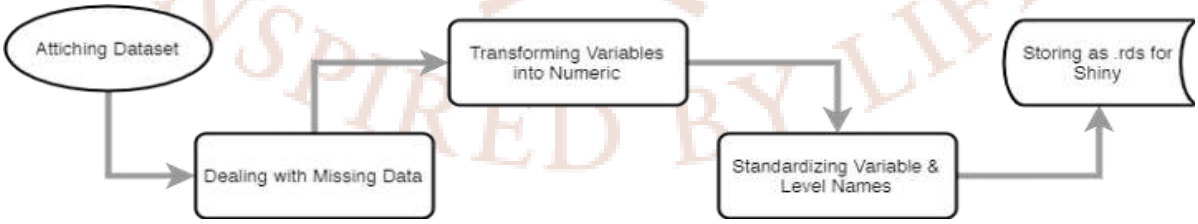


Fig 5: Workflow of Data Transformation

In the code given below (DataWrangling.R), we will essentially attach the dataset, remove NAs, fill missing values, alter university names to make it more readable, and then store the data frames in.rds format, which is much quicker to read at the subsequent steps.

```
## Read employment data
data_e <- read.csv('data/employment_data.csv', stringsAsFactors = FALSE)

## Check and remove NA. In the dataset we have 'na', let's change that with NA and then remove it
data_e[data_e == "na"] <- NA

table(is.na(data_e))
sapply(data_e, function(x)
  sum(is.na(x)))
```

Fig 6: Code

There are certain names that need cleaning if users look at the names of the school unique(data e$school) in the dataset, for instance:

```
data_e[data_e$school == "Singapore Institute of Technology (SIT)", "school"] <- "Singapore Institute of Technology"
data_e[data_e$school == "Singapore Institute of Technology -Trinity College Dublin", "school"] <- "Trinity College Dublin"
data_e[data_e$school == "SIT-University of Glasgow", "school"] <- "University of Glasgow"
```

Fig 7: Code

Numerical transformation of factor variables:

```
data_e <- data_e %>% modify_at(c(5:12), as.numeric)
```

Fig 8: Numerical transformation

Lastly, save the cleaned data frame with the Shiny.rds extension:

```
saveRDS(data_e, file = "data/employment_data.rds")
```

Fig 9: Cleaning Data

3. Shiny App structure

To run a Shiny App you need to install the package & import the library:

```
install.packages("Shiny")
library(Shiny)
```

Fig 10: Packages

Shiny apps have two crucial parts, which I refer to as the front-end user interface and the back-end server. R.

To design the front end of our web application, we use the ui.R programme.

This is a basic illustration of illustration of our dashboard's user interface:

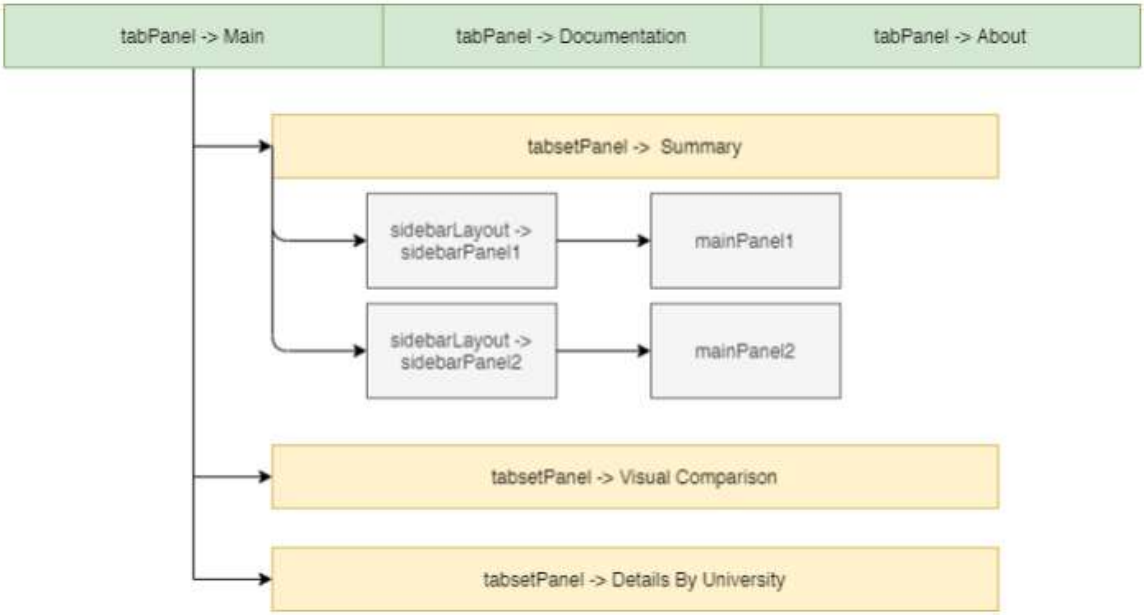


Fig 11: UI Dashboard

```
Shiny ui
library(...)

ui <- navbarPage("GraduateEmployment", theme = shinytheme("flatly"),
  tabPanel("Main",
    titlePanel(div(windowTitle = "GraduatEmploymentSG", img(src = "sg0.jpg", width = "100%", class = "bg"),)),

    Panel: Main>Summary
    tabsetPanel(type = "tabs",

      tabPanel("Summary",
        sidebarLayout(
          sidebarPanel(h3("Data by Year"),
            selectInput("checkYear", "Select Year", choices = list("2018", "2017", "2016", "2015", "2014", "2013"),selected = "2018")),

          mainPanel(
            tabsetPanel(type = "tabs",
              tabPanel("Ranking", tableOutput("datahead")),
              tabPanel("No. of Graduates", plotOutput(outputId = "piePlot"))),)),

      tabPanel()))
```

Fig 12: Code for front end

```
Shiny server
library(...)
data <- readRDS("data/employment_data.rds")
data_g <- readRDS("data/graduates_by_institutions.rds")
server <- function(session, input, output) {
  datahead section
  output$datahead <- renderPrint({
    data %>%
      filter(year == input$checkYear) %>%
      group_by(university) %>%
      select(university, employment_rate_overall, gross_monthly_median) %>%
      summarise_all(funs(mean)) %>%
      mutate_if(is.numeric, round, 0) %>%
      arrange(desc(employment_rate_overall)) %>%
      #summarise_at(.vars = names(.)[8], list(min, max)) %>%
      kable("html", col.names = c("University", "Employment Rate Overall (Avg %)", "Gross Monthly Median Income (Avg)")) %>%
      kable_styling(c("striped", "hover"), full_width = T)})
  piePlot section
  output$piePlot <- renderPlot({
    colmap <-
      c("#bdb2ff", # NUS "#ffc6ff", # NTU "#fffffc", # SMU "#33658A", # SIT "#3a506b", # SUTD "#577590", # SUSS "#43aa8b",
        # NIE "#90be6d", # SP "#f9c74f", # NP "#f8961e", # TP "#f3722c", # NAYANG POLY "#f94144", # RP "#ffadad",
        # NAFA DEG "#ffd6a5", # LAS DEG "#fdffb6", # NAFA DIP "#caffbf", # NAFA DEG "#a8dadc"  # ITE)
    data_g %>%
      filter(year == input$checkYear) %>%
      group_by(university) %>%
      tally(graduates) %>%
      ggplot(aes(x = "", y = n, fill = university)) +
      geom_bar(stat = "identity", width = 1, color = "black", size = 1) +
      theme_void() +
      theme(legend.position = "right", plot.title = element_text(hjust = 0.5, size = 14)) +
      coord_polar("y", start = 0) +
      scale_fill_manual(values = c(colmap)) +
      labs(title = "Universities by number of graduates") })
}
```

Fig 13: Server code

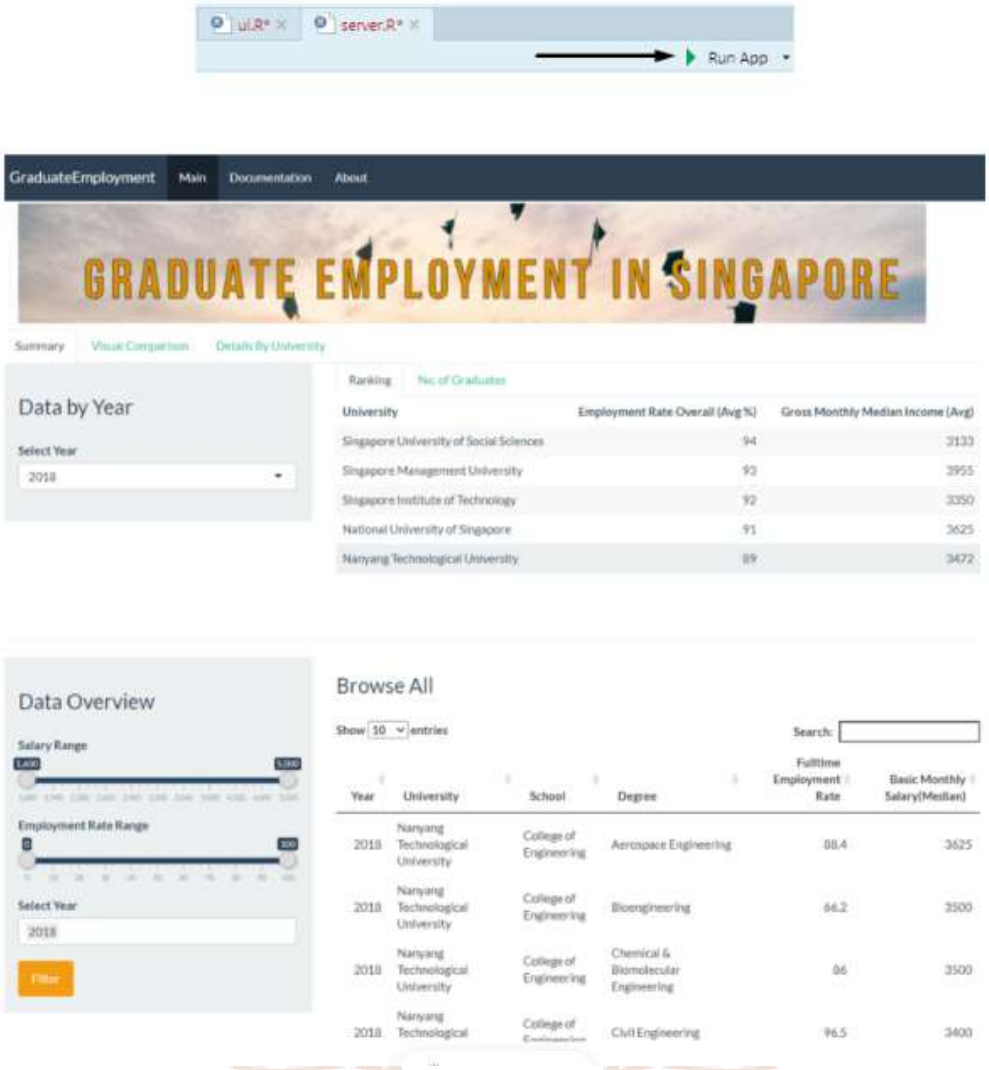


Fig 14: Result for the code

4. Visualizations

The majority of visualizations are created using Plotly, and Kable is used to present the data tables.

```
monthly <- data %>%
      filter(university == "National University of Singapore") %>%
      group_by(school) %>%
      summarise_at(.vars = names(.)[7:8], .funs = c(mean = "mean"))
colmap <-
      c("#2c3e50","#e67e22","#f1c40f","#e74c3c","#F97F51","#27ae60", "#2980b9","#86BBD8","#8e44ad",
      "#95a5a6","#f39c12","#d35400", "#c0392b","#bdc3c7","#D6A2E8","#25CCF7","#16a085")
p <- monthly %>%
      data.frame() %>%
      ggplot(., aes(x = reorder(school, basic_monthly_median_mean),
                y = basic_monthly_median_mean,fill = school)) +
      geom_bar(stat = "identity", width = 0.5, color = "black", size = 1) +
      scale_fill_manual(values = colmap) +
      theme_hc() +
      theme(legend.position = "none", axis.title.x = element_blank(), axis.title.y = element_blank())

ggplotly(p + coord_flip(), tooltip = ("basic_monthly_median_mean"))title = "Universities by number of graduates") })
}
```

Fig 15: Visualization code

Result:

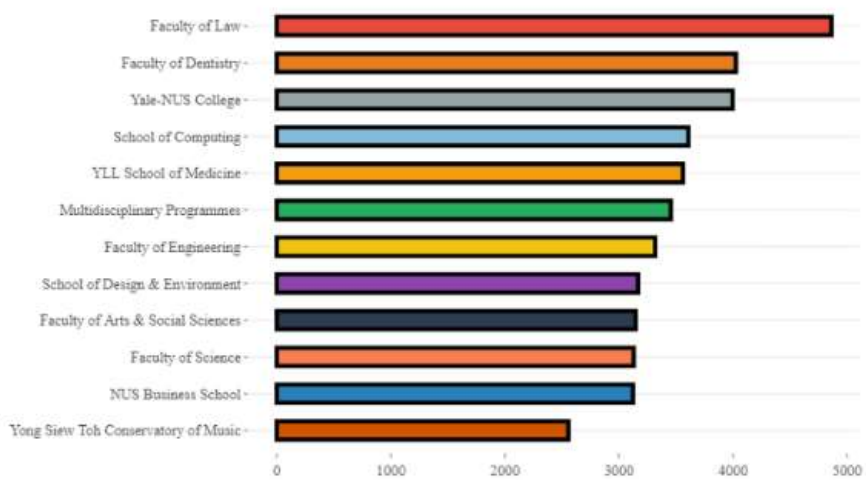


Fig 16: Monthly median Salary

5. Deployment of dashboard on https://www.shinyapps.io/ server.

Steps to create deploy on dashboard are:

By assuming that server.R and ui.R are located in the same folder.

1. Run the following programme from your R file:

   install.packages('rsconnect')

   library (rsconnect)

2. Create an account on shinyapps.io. Please be aware that all of your apps on shinyapps.io will use your account name as the domain name.
3. Obtain the website-generated token when you log in to shinyapps.io:

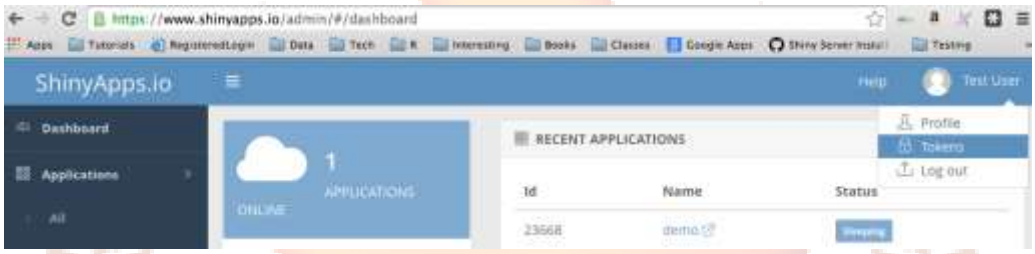


Fig 17: Website generated

4. Set up your account so the rsconnect package may utilise it. Bringing up the website Click the display button on the token page to start. A popup displaying the whole command to set up your account with the correct arguments for the rsconnect::setAccountInfo function will appear. To use this command, copy it to your clipboard, paste it into RStudio's command line, and press enter.

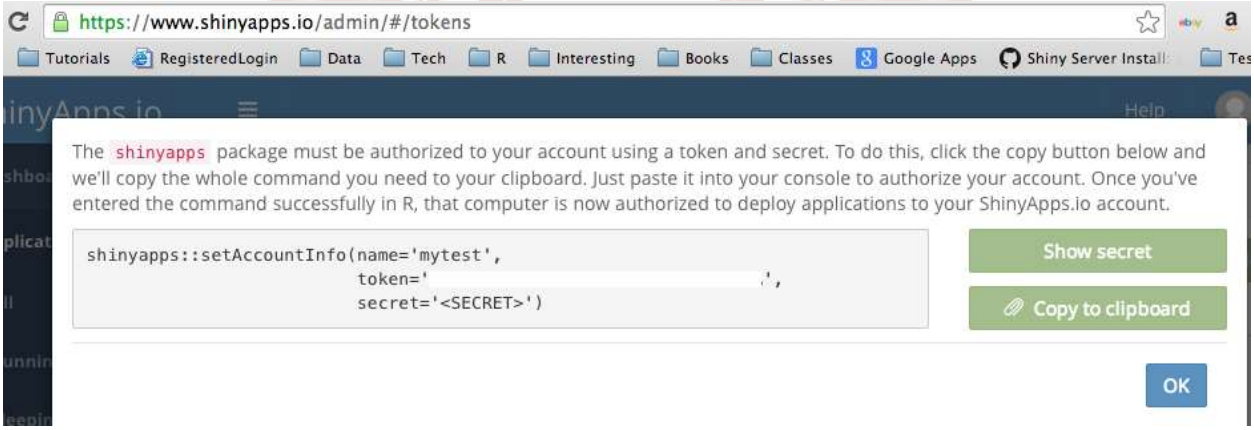


Fig 18: Setting up account

5. Release the app. Use the following codes to deploy your application:

   library(rsconnect)

   deployApp()

Next, in R, select the Publish button (from ui.R or server.R).

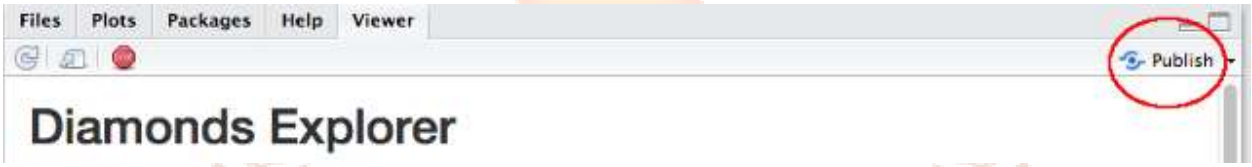


Fig 19: Publish

Congratulations when the deployment is complete! Your initial Shiny App has been released. You can publish the app again after making modifications to the server.R or ui.R files.

Self-Assessment Questions - 1

1. The majority of visualizations are created using _____, and _____ is used to present the data tables.
2. Codes to deploy your application _____ and _____.

3. ADDITION OF ADVANCED VISUALIZATIONS IN R-SHINY DASHBOARD

Data Visualization

Today's world has an exponentially growing amount of data, making it impossible to tell tales without them. Even while there are specialized tools available, such as Tableau, QlikView, and d3.js, nothing can substitute a modeling and statistics tool with strong visualization capabilities. Both feature engineering and any type of exploratory data analysis benefit greatly from it. R is a huge assistance in this regard.

To create visualizations and show data, R Programming provides a sufficient selection of built-in functions and libraries (such as ggplot2, leaflet, and lattice).In this there will be brief explanation about the basic and advanced visualizations in R-Shiny Dashboard.

Basic Visualization

- ➔ Histogram
- ➔ Bar Chart
- ➔ Line Chart

Advanced Visualization

- ➔ Heat Map
- ➔ Map Visualization
- ➔ Correlogram

BASIC VISUALIZATION

1. **HISTOGRAM**

   The most popular graph to represent continuous data is a histogram. It is a bar plot that shows the measurements' frequencies of appearance and counts the number of observations that fall within each interval. Additionally, the height is influenced by the ratio of frequency to interval width.

Example:The code below creates a function called binner, which takes a vector named var, and saves the histogram app as that. The function sends shinyApp its var parameter, which causes shinyApp to start an app that displays var.

```
binner <- function(var) {
  require(shiny)
  shinyApp(
    ui = fluidPage(
      sidebarLayout(
        sidebarPanel(sliderInput("n", "Bins", 5, 100, 20)),
        mainPanel(plotOutput("hist"))
      )
    ),
    server = function(input, output) {
      output$hist <- renderPlot(
        hist(var, breaks = input$n,
          col = "skyblue", border = "white")
      )
    }
  )
}
```

Fig 20 : Code to implement Histogram

Output:

binner(faithful$eruptions)

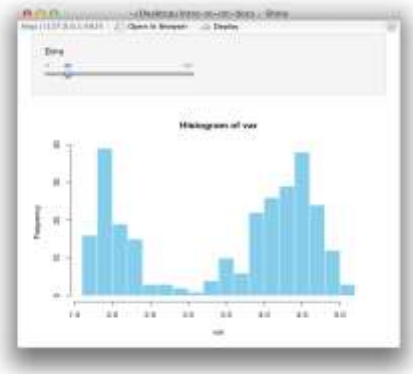


Fig 21: Histogram 1

binner(iris$Sepal.Length)

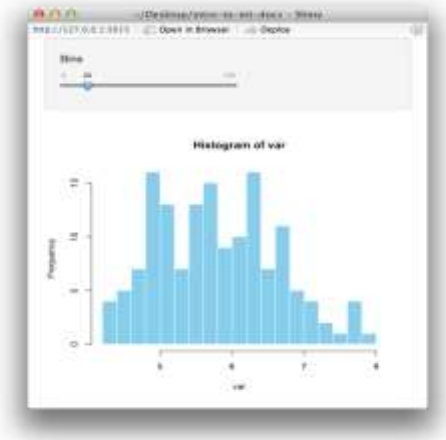


Fig 22: Histogram 2

2. BAR CHART

Rectangular bars with lengths proportionate to the values of the variables are used in bar charts to display data. Bar charts are made in R using the function barplot(). R could produce both vertical and horizontal bars in a bar chart. Each bar in a bar graph can have a distinct color.

Example:

```
ui <- fluidPage(

  sidebarLayout(
    sidebarPanel(

      p(style = "font-family:Impact",
        a("metab Expectation")),

      selectInput("var",label="Choose an method",choice=c("M0",
                                                          "M1",
                                                          "M2",
                                                          "M3",
                                                          "M4",
                                                          "M_Total"),
      selectInput(inputId = "Group", label = strong("Group"),
                  choices = unique(Aspartate$Group),
                  selected = "DM"),

      selectInput(inputId = "Timepoint", label = strong("Timepoint"),
                  choices = unique(Aspartate$Timepoint),
                  selected = 30),
    ),
```

Fig 23 : Code to implement Bar chart

Output:

Fig 24 : Bar chart

3. LINE CHART

A line graph, also known as a line plot or a line chart, connects each individual data point with a line. In a line graph, numbers are represented as a function of time.

Example:The code illustrates how to give each line on the chart a distinct color.

```
library(plotly)
library(dplyr)
library(shiny)
library(shinyWidgets)
library(readxl)
library(tidyr)

df1 <- data.frame(Month = rep(month.abb[1:12],10,replace = TRUE), Product = rep(LETTERS[1:10], each = 12),
                  Value = sample(c(0:300),120, replace = T),
                  stringsAsFactors = F)
df2 <- data.frame(Month = rep(month.abb[1:12],10,replace = TRUE), Product = rep(LETTERS[1:10], each = 12),
                  Value = sample(c(0:300),120, replace = T),
                  stringsAsFactors = F)

# UI
ui <- fluidPage(
  column(
    6,fluidRow(column(6, selectizeInput("All", "Year: 2018", multiple = T,choices = unique(df1$Product),
                                        options = list(maxItems = 5, placeholder = 'Choose a product:'))),
               column(6, selectizeInput("All2", "Year: 2019", multiple = T,choices = unique(df2$Product),
                                        options = list(maxItems = 5, placeholder = 'Choose a product:'))))
  ),
  column(
    12,fluidRow(column(12, plotlyOutput('plot'))
```

Fig 25 : Code to implement different color in line chart-1

```
# Server code
server <- function(input, output) {

  outVar <- reactive({
    df1 %>%
      filter(Product %in% input$All) %>%
      mutate(Product = paste(Product, "2018", sep = "_")) %>%
      arrange(Month) %>%
      droplevels()
  })

  output$plot <- renderPlotly({
    plot_ly(data=outVar(), x=~Month,  y = ~Value,
            type = 'scatter', mode = 'lines', legendgroup = "1",
            color = ~Product  , colors = c("grey","darkred","orange","black","purple")) %>%
      add_trace(data=outVar2(), x=~Month,  y = ~Value,
                type = 'scatter', mode = 'lines', legendgroup = "2",
                color = ~Product , colors = c('red','blue', 'yellow', 'green', "pink"))  %>%
      layout(legend = list(orientation = 'h'))
  })
}
shinyApp(ui = ui, server = server)
```

Fig 26 : Code to implement different color in line chart-1

Output:

Fig 27 : Line Chart

ADVANCED VISUALIZATION

1. **HEAT MAP**

   A heatmap is a graphical depiction of data that uses color coding to illustrate different values. Although heatmaps can be used for many different types of analytics, they are most frequently used to display user behavior on certain webpages or web page layouts.

   Example:By using iris dataset the Heat map will be visualized.

```
library(shiny)
library(heatmaply)
library(shinyHeatmaply)
runApp(system.file("shinyapp", package = "shinyHeatmaply"))
library(shinyHeatmaply)
#single data.frame
data(mtcars)
launch_heatmaply(mtcars)
#list
data(iris)
launch_heatmaply(list('Example1'=mtcars,'Example2'=iris))
```

Fig 28 : Code to implement different color in Heat map

Output:

Fig 29 : Heat Map

2. MAP VISUALIZATION

Through map visualisation, spatially pertinent data is examined, shown, and presented. This type of data expression is more comprehensible and transparent. The distribution or percentage of data in each area can be seen visually.

Example:The UScensus2010 was used to compile the dataset counties.rds, which contains demographic information for every county in the country.

```
ui <- fluidPage(titlePanel("censusVis"),
  sidebarLayout(
    sidebarPanel(
      helpText("Create demographic maps with
        information from the 2010 US Census."),
      selectInput("var", label = "Choose a variable to display", choices = c("Percent White", "Percent Black",
                                "Percent Hispanic", "Percent Asian"),selected = "Percent White"),
      sliderInput("range",
                  label = "Range of interest:",
                  min = 0, max = 100, value = c(0, 100))
    ),
     mainPanel(plotOutput("map"))
  )
)
# Server logic ----
server <- function(input, output) {
  output$map <- renderPlot({
    data <- switch(input$var, "Percent White" = counties$white, "Percent Black" = counties$black,
    "Percent Hispanic" = counties$hispanic,"Percent Asian" = counties$asian)
    percent_map(var = data, color = ?, legend.title = ?, max = ?, min = ?)
  })
}
shinyApp(ui, server)
```

Fig 30 : Code to implement Map visualization

Output:

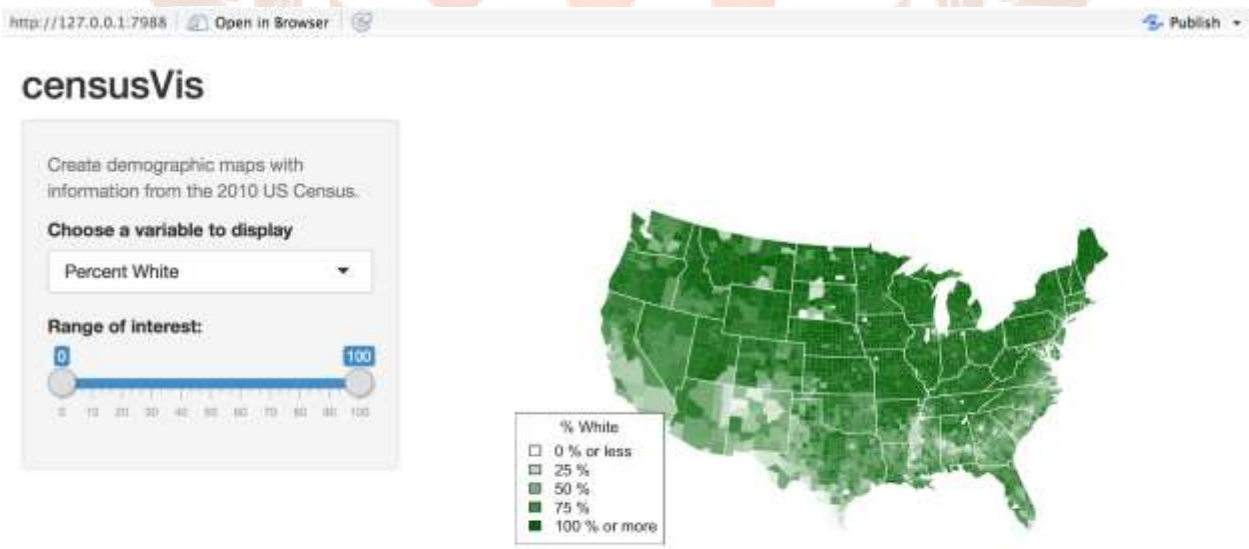


Fig 31 : Map Visualization

3. CORRELOGRAM

A correlation matrix is represented as a correlogram. Highlighting the variables in a data table that are highly connected is quite helpful. According to their values, correlation coefficients are coloured in this graph. The correlation matrix can also be rearranged in accordance with how strongly two variables are related.

Example:By using Iris dataset the Correlogram has been visualized.

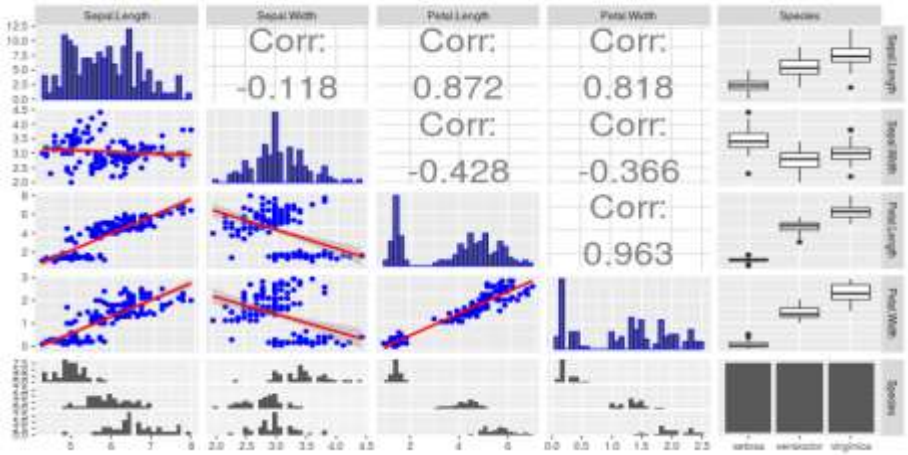


Fig 32 : Correlogram

Self-Assessment Questions - 2

3. _____ provides a sufficient selection of built-in functions and libraries (such as ggplot2, leaflet, and lattice).
4. A _____ is represented as a correlogram.

4. ACTIVITY

Activity A

In the Chinese city of Wuhan, incidences of severe respiratory sickness started to be reported in December 2019. These were brought on by a novel coronavirus, and the illness is now generally known as COVID-19. Midway through January, the number of COVID-19 cases began to increase more swiftly, and the virus soon expanded outside of China. Since then, this story has quickly developed, and every day we are presented with unsettling stories about the outbreak's current situation.

These headlines can be challenging to understand on their own. How quickly is the virus circulating? Work being done to control the disease? How does the current scenario differ from past epidemics?

……

The objective is to develop a visualization which updates data like number of affected cases,deaths and mostly affected countries based on the mapping date.

5. SUMMARY

R-based data specialists can easily incorporate Shiny into their development workflow. Shiny dashboards are adaptable, dynamic, and simple to tailor to each customer's unique requirements. This is partially attributable to the web framework's support for web technologies including HTML, CSS, SCSS, JavaScript, and others. Writing R codes to plot graphs repeatedly can become very tiresome. Additionally, making an interactive visualization for story narration is very challenging. Therefore, the issues can be easily fixed by quickly building interactive charts in R using Shiny. Utilizing Shiny makes it possible to employ modularized codes and functions, quickly prototype ideas, and easily manage dashboards using smaller components.

6. GLOSSARY

Data wrangling - Eliminating errors and integrating complex data sets to make them more accessible and understandable.

Correlogram - Displays the correlations between each pair of variables.

7. CONCEPT MAP

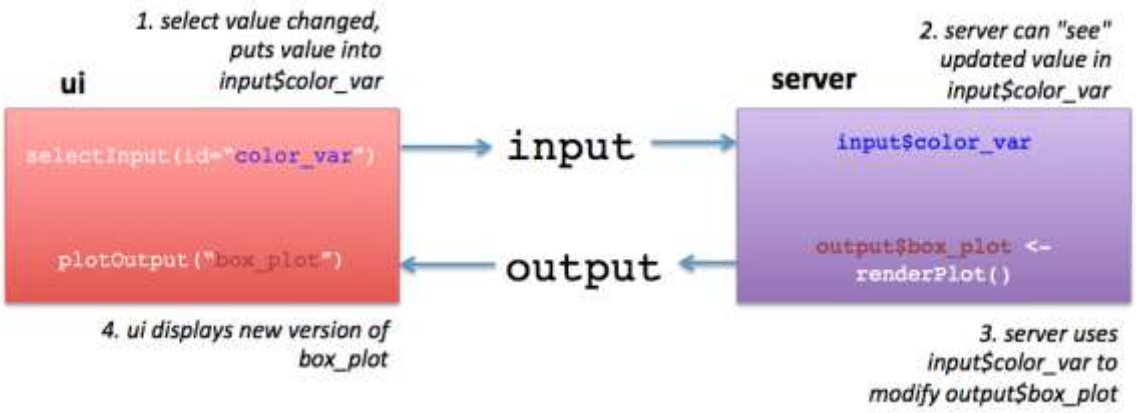


Fig 33 : MAP TO CONNECT UI TO THE SERVER

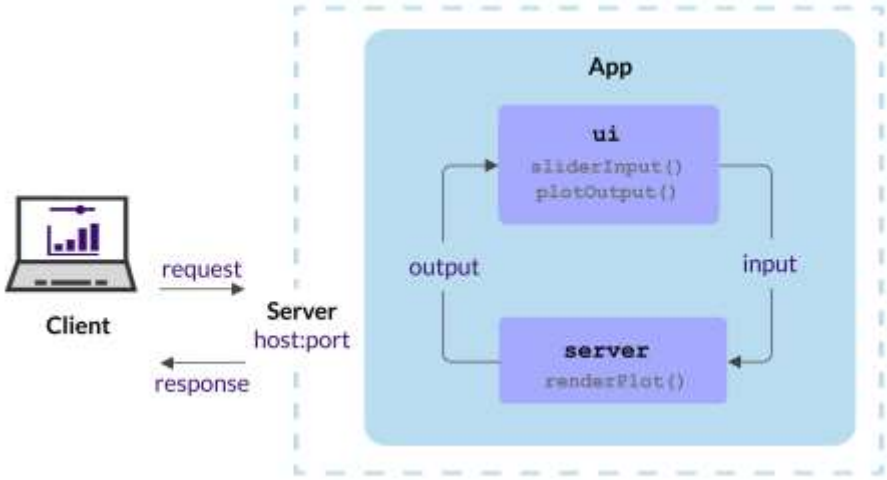


Fig 34 :ANATOMY OF SHINY APPLICATION

8. STUDY NOTES & DID YOU KNOW

Study Notes – Shiny applets like the Covid Tracker and RadaR, an R-based interactive tool for rapid analysis of diagnostic and antimicrobial patterns, provide an example.

Study Notes – With the help of the shiny package, you can quickly create a user interface (UI) and use R code to update the plots and analyses that are displayed to the user in response to their selection of various UI options.

9. CASE STUDY

a) MRI images in Shiny

The use of R-Shiny Visualizations in medical imaging has significant positive effects on the healthcare industry. A significant study in this field is Big Data Analytics in Healthcare, which was published in BioMed Research International. The study lists several common imaging methods, such as computed tomography, computed radiography, mammography, and magnetic resonance imaging (MRI). The disparity in these images' modality, resolution, and dimensions is addressed using a variety of techniques. To enhance image quality, more effectively extract data from photos, and offer the most accurate interpretation, many more are currently being developed. By learning from prior cases and then suggesting better treatment options, the deep-learning based algorithms improve diagnostic accuracy.

Using only R shine mechanisms, this software interactively visualizes 3D MRI scans.

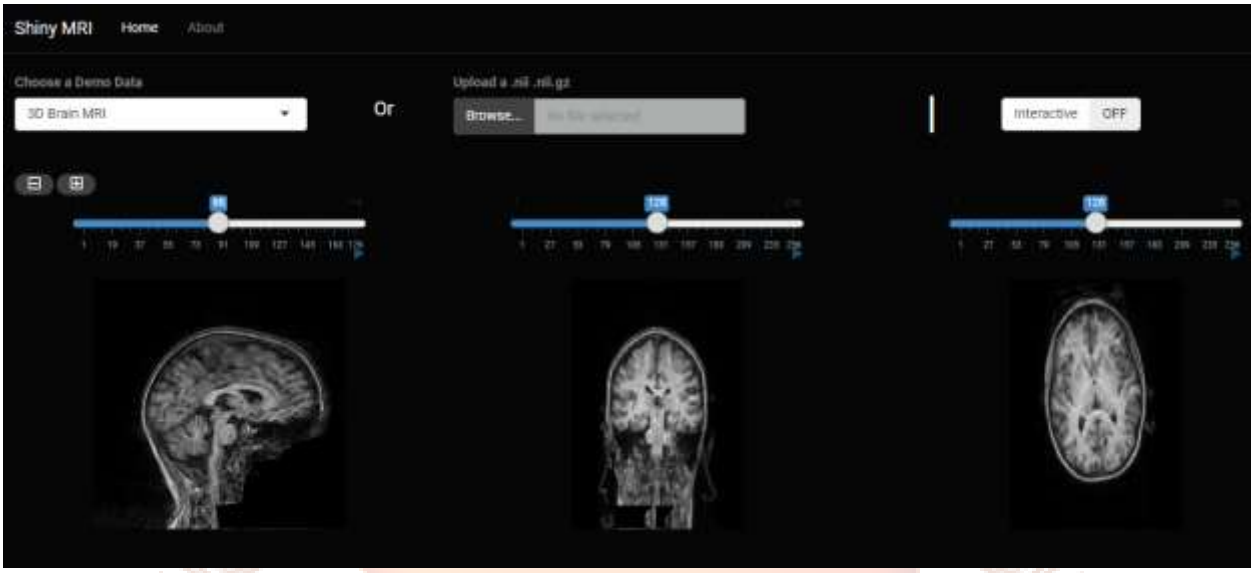


Fig 35 : 3-D Brain MRI

Fig 36 : 4-D Brain MRI

10. TERMINAL QUESTIONS

Short Answers Type:

1. Draw a workflow diagram for data transformation.
2. Define Histogram.

Long Answers Type:

1. Describe the steps involved in creating a dashboard with code.
2. Explain any two of the advanced visualizations with the code using R-Shiny.

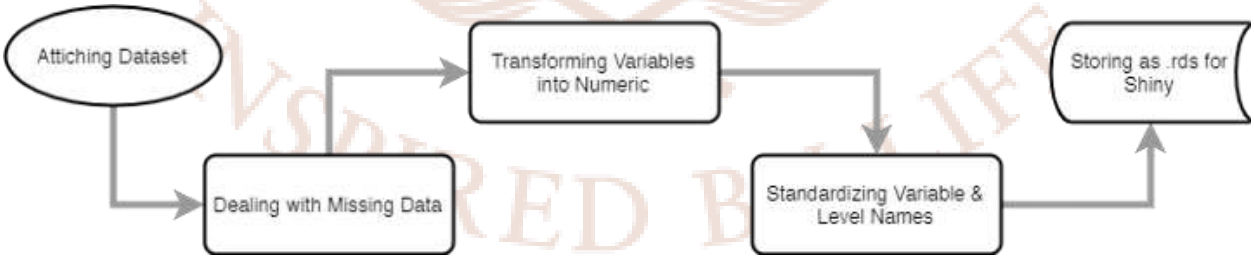
11. SELF ASSESSMENT ANSWERS

1. Plotly and Kable
2. library(rsconnect) and deployApp()
3. R Programming
4. Correlation matrix

12. TERMINAL QUESTION ANSWERS

Short Answer Type:

1.



2. The most popular graph to represent continuous data is a histogram. It is a bar plot that shows the measurements' frequencies of appearance and counts the number of observations that fall within each interval. Additionally, the height is influenced by the ratio of frequency to interval width.

Long Answer Type:

1. **Contents involved to create a Shiny Dashboard**
   - Data sources
   - Data transformation
   - Shiny App structure
   - Visualizations
   - Dashboard deployment on the shinyapps.io website server.

Let us get to know each topic in detail with an example

1. **Data Source:-**

   In this study, two datasets will be used.

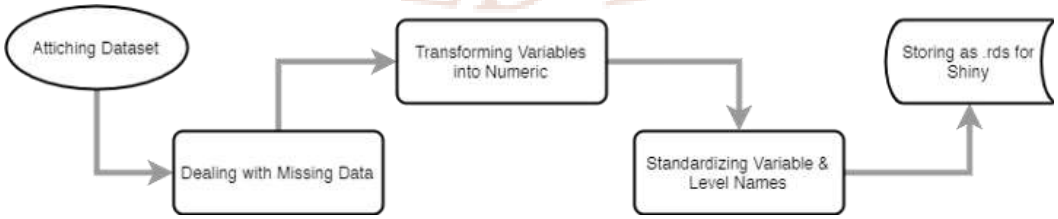
→ Data from the Graduate Employment Survey is available at data.gov.sg/ges.

→ Users can find the dataset for grads by university here: data.gov.sg/gra.

As previously indicated, the analysis's primary metrics are:

- Rate of Employment
- Gross Monthly Income

2. **Data transformation**



Workflow of Data Transformation

In the code given below (DataWrangling.R), we will essentially attach the dataset, remove NAs, fill missing values, alter university names to make it more readable, and then store the data frames in.rds format, which is much quicker to read at the subsequent steps.

```
## Read employment data
data_e <- read.csv('data/employment_data.csv', stringsAsFactors = FALSE)

## Check and remove NA. In the dataset we have 'na', let's change that with NA and then remove it
data_e[data_e == "na"] <- NA

table(is.na(data_e))
sapply(data_e, function(x)
  sum(is.na(x)))
```

There are certain names that need cleaning if users look at the names of the school unique(data e$school) in the dataset, for instance:

```
data_e[data_e$school == "Singapore Institute of Technology (SIT)", "school"] <- "Singapore Institute of Technology"
data_e[data_e$school == "Singapore Institute of Technology -Trinity College Dublin", "school"] <- "Trinity College Dublin"
data_e[data_e$school == "SIT-University of Glasgow", "school"] <- "University of Glasgow"
```

Numerical transformation of factor variables:

```
data_e <- data_e %>% modify_at(c(5:12), as.numeric)
```

Numerical transformation

Lastly, save the cleaned data frame with the Shiny.rds extension:

```
saveRDS(data_e, file = "data/employment_data.rds")
```

Cleaning Data

3. **Shiny App structure**

You must install the package and import the library in order to start a Shiny app:

```
install.packages("Shiny")
library(Shiny)
```

Packages

Shiny apps have two crucial parts, which I refer to as the front-end user interface and the back-end server. R.

To design the front end of our web application, we use the ui.R programme.

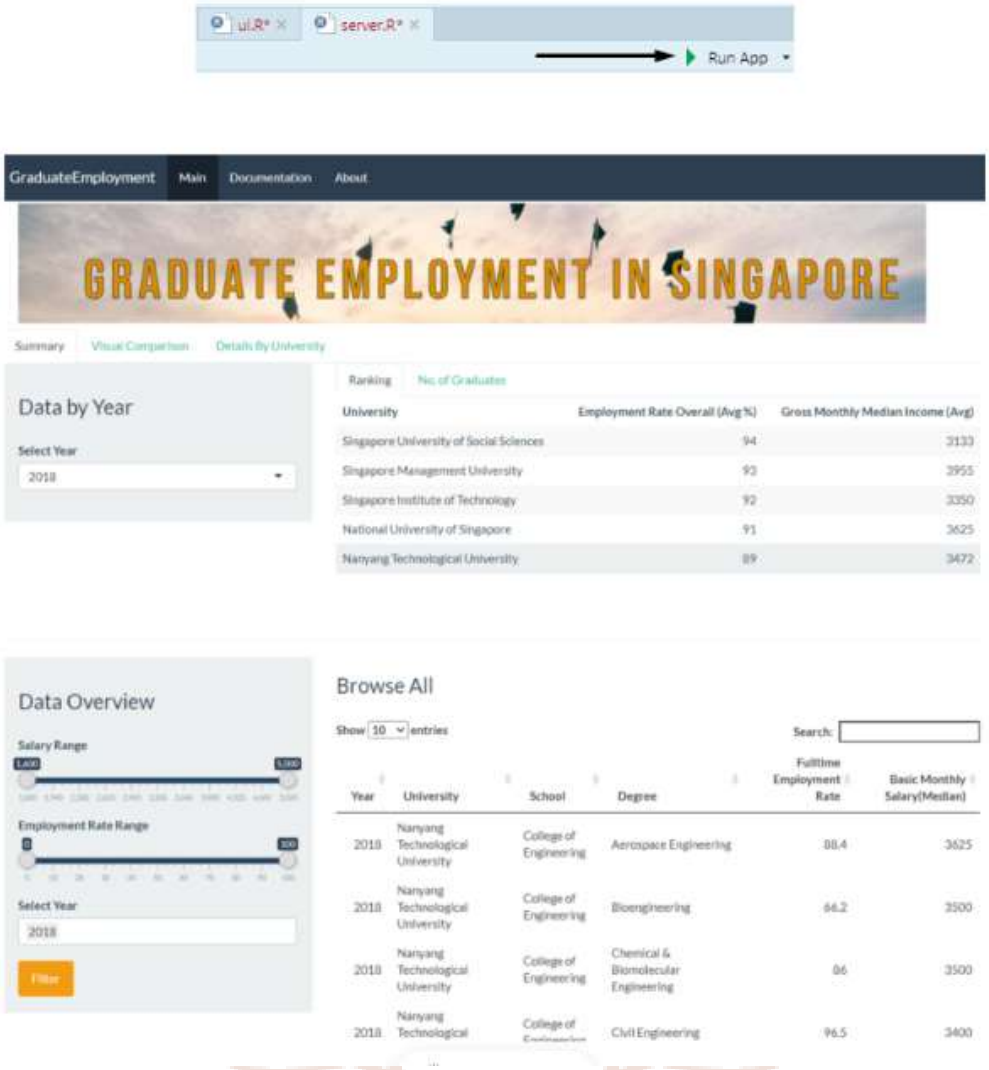
This is a basic illustration of illustration of our dashboard's user interface:

```
Shiny ui
library(...)

ui <- navbarPage("GraduateEmployment", theme = shinytheme("flatly"),
  tabPanel("Main",
    titlePanel(div(windowTitle = "GraduatEmploymentSG", img(src = "sg0.jpg", width = "100%", class = "bg"),)),

    Panel: Main>Summary
    tabsetPanel(type = "tabs",

      tabPanel("Summary",
        sidebarLayout(
          sidebarPanel(h3("Data by Year"),
            selectInput("checkYear", "Select Year", choices = list("2018", "2017", "2016", "2015", "2014", "2013"),selected = "2018")),

          mainPanel(
            tabsetPanel(type = "tabs",
              tabPanel("Ranking", tableOutput("datahead")),
              tabPanel("No. of Graduates", plotOutput(outputId = "piePlot"))),)),

      tabPanel()))
```

Code for front end

```r
Shiny server
library(...)
data <- readRDS("data/employment_data.rds")
data_g <- readRDS("data/graduates_by_institutions.rds")
server <- function(session, input, output) {
  datahead section
  output$datahead <- renderPrint({
    data %>%
      filter(year == input$checkYear) %>%
      group_by(university) %>%
      select(university, employment_rate_overall, gross_monthly_median) %>%
      summarise_all(funs(mean)) %>%
      mutate_if(is.numeric, round, 0) %>%
      arrange(desc(employment_rate_overall)) %>%
      #summarise_at(.vars = names(.)[8], list(min, max)) %>%
      kable("html", col.names = c("University", "Employment Rate Overall (Avg %)", "Gross Monthly Median Income (Avg)")) %>%
      kable_styling(c("striped", "hover"), full_width = T)})
  piePlot section
  output$piePlot <- renderPlot({
    colmap <-
      c("#bdb2ff", # NUS "#ffc6ff", # NTU "#fffffc", # SMU "#33658A", # SIT "#3a506b", # SUTD "#577590", # SUSS "#43aa8b",
        # NIE "#90be6d", # SP "#f9c74f", # NP "#f8961e", # TP "#f3722c", # NAYANG POLY "#f94144", # RP "#ffadad",
        # NAFA DEG "#ffd6a5", # LAS DEG "#fdffb6", # NAFA DIP "#caffbf", # NAFA DEG "#a8dadc"  # ITE)
    data_g %>%
      filter(year == input$checkYear) %>%
      group_by(university) %>%
      tally(graduates) %>%
      ggplot(aes(x = "", y = n, fill = university)) +
      geom_bar(stat = "identity", width = 1, color = "black", size = 1) +
      theme_void() +
      theme(legend.position = "right", plot.title = element_text(hjust = 0.5, size = 14)) +
      coord_polar("y", start = 0) +
      scale_fill_manual(values = c(colmap)) +
      labs(title = "Universities by number of graduates") })
}
```

Server code



Result for the code

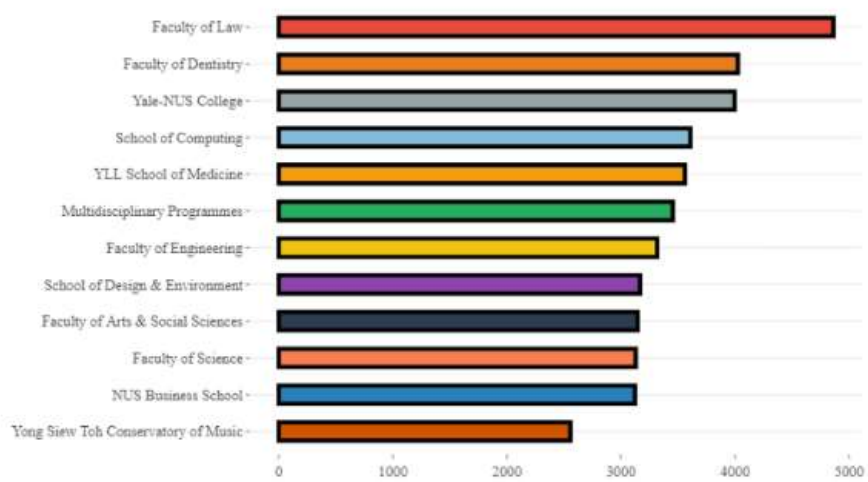
4. Visualizations

The majority of visualizations are created using Plotly, and Kable is used to present the data tables.

```
monthly <- data %>%
      filter(university == "National University of Singapore") %>%
      group_by(school) %>%
      summarise_at(.vars = names(.)[7:8], .funs = c(mean = "mean"))
colmap <-
      c("#2c3e50","#e67e22","#f1c40f","#e74c3c","#F97F51","#27ae60", "#2980b9","#86BBD8","#8e44ad",
      "#95a5a6","#f39c12","#d35400", "#c0392b","#bdc3c7","#D6A2E8","#25CCF7","#16a085")
p <- monthly %>%
      data.frame() %>%
      ggplot(., aes(x = reorder(school, basic_monthly_median_mean),
              y = basic_monthly_median_mean,fill = school)) +
      geom_bar(stat = "identity", width = 0.5, color = "black", size = 1) +
      scale_fill_manual(values = colmap) +
      theme_hc() +
      theme(legend.position = "none", axis.title.x = element_blank(), axis.title.y = element_blank())

ggplotly(p + coord_flip(), tooltip = ("basic_monthly_median_mean"))title = "Universities by number of graduates") })
}
```

Visualization code

Result:



Monthly median Salary

5. **Deployment of dashboard on https://www.shinyapps.io/ server.**

   Steps to create deploy on dashboard are:

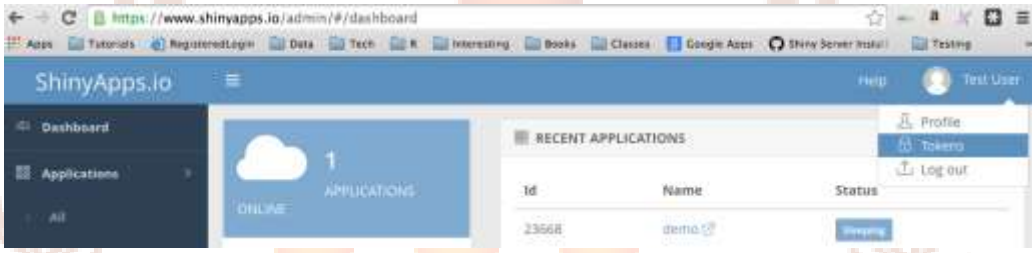
   By assuming that server.R and ui.R are located in the same folder.

1. Run the following programme from your R file:

   install.packages('rsconnect')

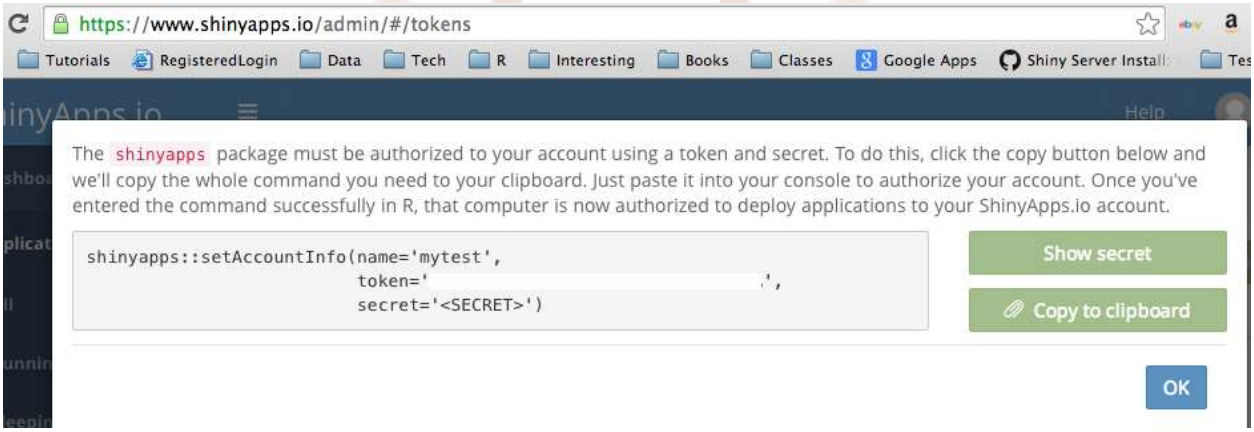
   library (rsconnect)

2. Create an account on shinyapps.io. Please be aware that all of your apps on shinyapps.io will use your account name as the domain name.
3. Obtain the website-generated token when you log in to shinyapps.io:



Website generated

4. Set up your account so the rsconnect package may utilize it. Bringing up the website Click the display button on the token page to start. A popup displaying the whole command to set up your account with the correct arguments for the rsconnect::setAccountInfo function will appear. To use this command, copy it to your clipboard, paste it into RStudio's command line, and press enter.



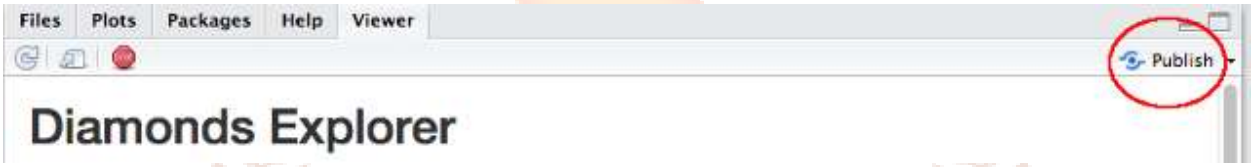
Setting up account

5. Release the app. Use the following codes to deploy your application:

   library(rsconnect)

   deployApp()

   Next, in R, select the Publish button (from ui.R or server.R).



Publish

Congratulations when the deployment is complete! Your initial Shiny App has been released. You can publish the app again after making modifications to the server.R or ui.R files.

2. **ADVANCED VISUALIZATION**
1. **HEAT MAP**

   A heatmap is a graphical depiction of data that uses color coding to illustrate different values. Although heatmaps can be used for many different types of analytics, they are most frequently used to display user behavior on certain webpages or web page layouts.

   Example:By using iris dataset the Heat map will be visualized.

```
library(shiny)
library(heatmaply)
library(shinyHeatmaply)
runApp(system.file("shinyapp", package = "shinyHeatmaply"))
library(shinyHeatmaply)
#single data.frame
data(mtcars)
launch_heatmaply(mtcars)
#list
data(iris)
launch_heatmaply(list('Example1'=mtcars,'Example2'=iris))
```

Code to implement different color in Heat map

Output:

Heat Map

2. MAP VISUALIZATION

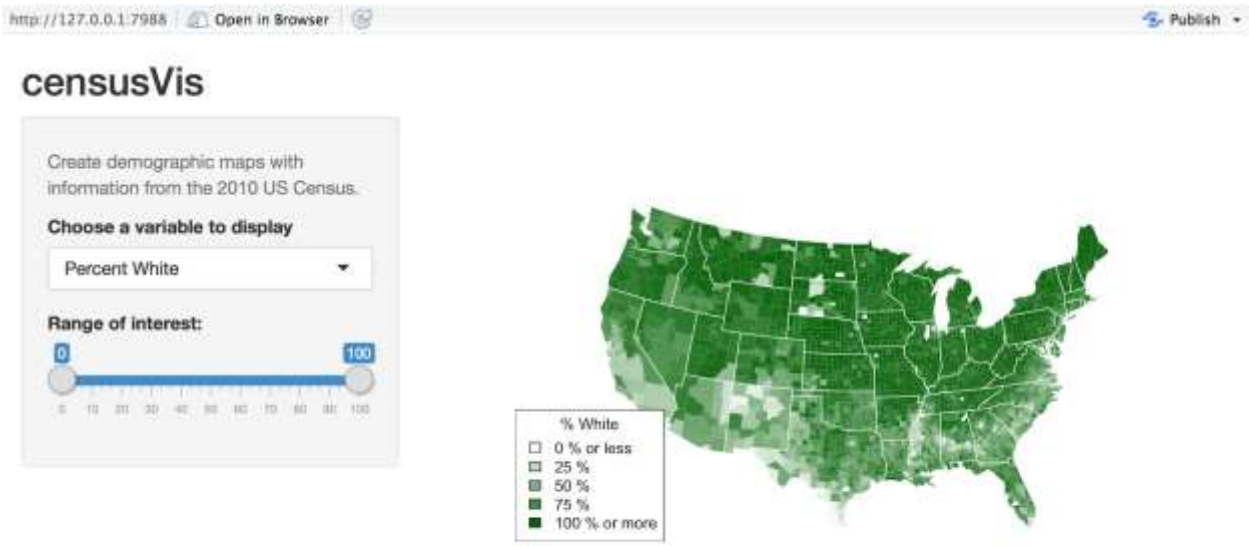
Through map visualisation, spatially pertinent data is examined, shown, and presented. This type of data expression is more comprehensible and transparent. The distribution or percentage of data in each area can be seen visually.

Example:The UScensus2010 was used to compile the dataset counties.rds, which contains demographic information for every county in the country.

```
ui <- fluidPage(titlePanel("censusVis"),
  sidebarLayout(
    sidebarPanel(
      helpText("Create demographic maps with
        information from the 2010 US Census."),
      selectInput("var", label = "Choose a variable to display", choices = c("Percent White", "Percent Black",
                                "Percent Hispanic", "Percent Asian"),selected = "Percent White"),
      sliderInput("range",
                  label = "Range of interest:",
                  min = 0, max = 100, value = c(0, 100))
    ),
     mainPanel(plotOutput("map"))
  )
)
# Server logic ----
server <- function(input, output) {
  output$map <- renderPlot({
    data <- switch(input$var, "Percent White" = counties$white, "Percent Black" = counties$black,
    "Percent Hispanic" = counties$hispanic,"Percent Asian" = counties$asian)
    percent_map(var = data, color = ?, legend.title = ?, max = ?, min = ?)
  })
}
shinyApp(ui, server)
```

Code to implement Map visualization

Output:



Map Visualization

13. REFERENCE

- https://appsilon.com/how-i-built-an-interactive-shiny-dashboard-in-2-days-without-any-experience-in-r/
- https://towardsdatascience.com/end-to-end-dashboard-in-r-shiny-app-64c40d0351d8
- https://avikarn.com/2020-05-26-correlation_shiny/
- https://shiny.rstudio.com/gallery/covid19-tracker.html